

GraphLang: an agent-optimized graph query and construction language, and its v0 engine

Design, implementation, and evaluation report

GraphLang project

August 25, 2026

Abstract

Language models write graph database queries badly, and they fail in structural, predictable ways: reversed relationship directions, hallucinated schema elements, implicit-grouping mistakes, and long-range syntax errors. GraphLang is a graph query and construction language that removes each documented failure mode by construction rather than by prompting around it, paired with a write surface no existing query language offers agents: provenance-carrying structural writes with receipts, system-detected and agent-resolved deduplication, agent-controlled granularity with full lineage, and queryable per-node health. This report documents the complete v0: the data model, every language construct with its exact semantics, the verify-before-execute pipeline, the single-process engine whose write-ahead-log and immutable-run architecture preserves the shape of the planned distributed system, and the development methodology (test-first with an adversarial review loop that surfaced thirteen confirmed defects before evaluation). On a 60-question CypherBench slice with both conditions running the same models live, GraphLang reaches 98.3% execution accuracy with Claude Haiku 4.5 against 73.3% for text-to-Cypher with the same model; on multi-hop questions the margin is 96.0% to 56.0%. The language has no training corpus; it is taught entirely in context.

Contents

1	Introduction	3
2	Design principles	3
3	The language	4
3.1	Data model	4
3.2	Program structure	5
3.3	Reads	5
3.4	Results, budgets, and continuation	6
3.5	Writes and receipts	6
3.6	Granularity: refine and compact	7
3.7	Schema evolution: derive class	7
3.8	Deduplication pipeline	8
3.9	Health and the maintenance loop	8
3.10	Verification and errors	8

3.11 Sessions and binding lifetime	9
4 Implementation	9
4.1 Architecture	9
4.2 Parser	9
4.3 Verifier	10
4.4 Storage engine	10
4.5 Read executor	10
4.6 Write executor, dedup, health	10
4.7 Session and CLI	11
5 Development methodology	11
6 Evaluation	11
6.1 Setup	11
6.2 Results	12
6.3 The evaluation fed the language	13
6.4 Error analysis and threats to validity	13
7 Will better models not simply fix Cypher?	13
7.1 Scaling has had two years and has not delivered	13
7.2 The reason it stays flat is structural: corpus, not capacity	14
7.3 When syntax errors fall, the surviving errors are the dangerous ones	14
7.4 A perfect Cypher generator still lacks the agent surface	14
7.5 Even for a perfect generator, the economics invert	15
8 Limitations and future work	15
A Complete grammar	15
B A construction session, end to end	16

1 Introduction

When an AI agent needs to query a graph database today, it writes Cypher, the query language of Neo4j, or one of its relatives. The best frontier models get roughly forty percent of realistic Cypher queries wrong: Claude 3.5 Sonnet reaches 61.6% execution accuracy on CypherBench and GPT-4o 60.2%, with sharply worse results on multi-hop questions [arXiv 2412.18702]. The errors are not noise. The benchmark’s own error taxonomy shows the same handful of failure modes over and over: the model reverses the direction of a relationship arrow, invents a label or property the schema does not have, writes a pattern that is syntactically valid but means something other than the question, mishandles Cypher’s implicit grouping, or loses a bracket across a long span.

Production agent-memory systems responded by hiding the graph entirely behind a few fixed retrieval tools. That works, but it forfeits expressivity: every new question shape needs a new hand-written tool. GraphLang pursues the alternative the research points at but nobody had built: redesign the language so each failure mode is impossible to write, or is caught by a verifier before anything executes. The design evidence (assembled in an earlier research phase and frozen as eighteen explicit decisions) comes from the Anka result, where a purpose-built DSL with zero pretraining exposure beat Python by forty points on multi-step tasks purely by removing error-prone degrees of freedom [arXiv 2512.23214], from grammar prompting [arXiv 2305.19234], and from the token-economics literature on how result formats affect model accuracy [arXiv 2310.04560, 2504.07087, 2410.01985].

GraphLang goes beyond querying. Its target workload is agents *building and maintaining* graphs, so the language owns the write side: structural writes with provenance and receipts, deduplication as a first-class dialogue between system and agent, granularity as an agent-controlled property of nodes, temporal retirement, gated schema evolution, and per-node health signals that make the graph’s own damage a queryable dataset.

This report covers four things: the language itself, construct by construct, with exact semantics (Section 3); the v0 implementation, module by module (Section 4); the development methodology, including the adversarial review loop and what it caught (Section 5); and the benchmark evaluation with its failure analysis, including three language changes the evaluation itself forced (Section 6). The complete grammar is Appendix A; a worked construction session is Appendix B.

Plain words

Cypher: Neo4j’s graph query language, the one all text-to-query benchmarks target. **Execution accuracy:** the fraction of generated queries whose *answers* match the gold answers when run, the strictest common metric. **Schema:** the catalogue of allowed node types, edge types, and properties. **DSL:** a small language built for one job. **Grammar prompting:** teaching a model a new language by placing a compact grammar plus worked examples in its context window.

2 Design principles

Nine rules, each traceable to a measured failure mode, govern every construct in the language:

1. **One line, one operation, one name.** Every statement performs a single operation and binds its result to an explicit name. Chaining and shadowing caused 69% of pipeline failures in the Anka study; named intermediates eliminate the class.
2. **No direction glyphs.** Edges are traversed by naming the destination *role*. Direction cannot be written, so it cannot be reversed.

3. **Canonical forms.** Exactly one spelling per operation. No optional shorthands, no second aggregation syntax. Two correct agents produce byte-identical queries.
4. **Explicit staged aggregation.** Grouping is its own line; editing a return list can never silently change what is counted.
5. **Line-local syntax.** Keyword-delimited, newline-terminated. No delimiter opens on one line and closes on another.
6. **Verify before execute.** The whole program is checked against the live schema first. Errors name the line, suggest alternatives, and state that nothing ran. Partial execution never happens.
7. **Words over symbols.** Verbose keywords ride the language prior; symbol-dense operators concentrate errors. Even arithmetic is spelled with words (`minus`, `same`).
8. **The result format is part of the language.** Results are token-budgeted, count-declared, incident-encoded, and never silently truncated.
9. **Receipts, not row counts.** Every write returns a structured receipt carrying what changed, which guards ran, and what the system noticed (duplicate candidates above all). The receipt is the system's half of the conversation.

3 The language

3.1 Data model

A graph is a set of **nodes** and **edges** governed by a schema. Every node belongs to a **class** (for example `supplier`); a class declares typed properties (`str`, `int`, `float`, `bool`) and may derive from a base class, inheriting its properties. Every edge belongs to an **edge type** that declares exactly two named **roles**, each bound to a class:

```
classes: product{name, launch_year} part{name, unit_cost}
         supplier{name, country}
edges:   uses(whole: product, component: part)
         supplied_by(item: part, source: supplier)
```

Roles are the load-bearing idea. The edge type `supplied_by` does not have a direction; it has an `item` end and a `source` end. A query traverses it by naming the role it wants to arrive at. When the two ends have different classes, asking to arrive at the wrong role is a type error the verifier rejects with a corrective message. When both ends share a class (`manages(manager: person, report: person)`), the choice is genuinely semantic, but it is now carried by a meaningful word rather than an arrow glyph.

Nodes additionally carry a **granularity state**, one of `blob` (an opaque payload such as a whole ingested table), `composite` (has structured children), or `atom` (a single fact). Classes declare which states they allow. Moving between grains is done by agents with explicit verbs (Section 3.6), and the system keeps **lineage**: every derived node knows its origin, and every merge or refinement is recorded with enough state to unwind it.

Nodes have stable ids of the form `#p-142` (class-initial prefix, monotonic counter). When two nodes merge, the absorbed id becomes a permanent alias of the survivor, so references never dangle.

3.2 Program structure

A program is a sequence of statements, one per *logical line*. A physical line starting with whitespace continues the previous logical line, which keeps long writes readable. Comments are a `#` followed by a space (a node id such as `#p-71002` on a continuation line is never mistaken for a comment). There are no brackets to balance beyond single-line property maps, no statement terminators, and no alternative spellings.

3.3 Reads

A read builds one **binding table**, in the style of SPARQL solution mappings. Each `as NAME` adds a column; each row is one consistent assignment of nodes and values to columns.

find seeds the table with one row per matching node:

```
find product where launch_year > 2024 as recent
```

The target is a class name or one of three special targets: **nodes** (any class), **dup_candidates** (the deduplication ledger), and **class** (schema introspection). A second **find** in the same query cross-joins its matches with the existing rows, which is how comparison questions bind two entities side by side.

Conditions are chains of clauses joined by **and** and **or** (with **and** binding tighter), over the operators `=`, `!=`, `>`, `>=`, `<`, `<=`, and **contains**. String comparison is case-insensitive and accent-insensitive (casefold plus Unicode NFKD accent stripping): agents transliterate names, writing "Jose Calderon" for José Calderón, and a query that silently matches nothing is a worse failure than a forgiving match. The tradeoff (two nodes whose names differ only by case would both match) is documented, and it is precisely the situation the dedup pipeline flags for resolution.

follow extends every row across an edge type, arriving at a named role, optionally filtering the arrival node:

```
follow recent uses component where unit_cost > 3 as parts
follow parts supplied_by source as sups
```

For each row, for each qualifying edge incident to the row's source node, a new row binds the arrival node; rows with no match are dropped. GraphLang uses **trail semantics**: within one row, the same edge *instance* is never traversed twice. Nodes may repeat, edges may not. This matches the uniqueness rule of a Cypher **MATCH**, so questions like "other teams in the same division as X" naturally exclude X, and benchmark gold answers agree.

group by declares a grouping key, and aggregation verbs consume it:

```
group by sups as g
count distinct g.parts as n_parts
```

group by sups groups by node *identity*: two suppliers who share a name stay separate. Grouping by a value is a visibly different spelling (**group by sups.country**), so the identity-versus-value trap that silently corrupts SQL and Cypher aggregations is an explicit choice here. The aggregation verbs are **count**, **sum**, **avg**, **min**, **max**, each with an optional **distinct** that deduplicates *bindings* before property extraction (so **avg distinct g.p.height_cm** averages over distinct players, not distinct heights). Members whose property is null are skipped. An aggregate applied to a plain (ungrouped) binding is a **global aggregate** and collapses the table to a single row: **count distinct v as n** answers "how many venues are there" with no artificial grouping step.

compute performs scalar arithmetic or equality between two bound values, with word operators:

```
compute p1.height_cm minus p2.height_cm as diff
compute p1.handedness same p2.handedness as answer
```

`same` yields a boolean and refuses to compare an unset value (both operands missing must not read as “equal”). The numeric operators are `plus`, `minus`, `times`, `over`.

`return` ends the query:

```
return sups.name, n_parts order by n_parts desc limit 20 budget 2000 tokens
```

`order by` may sort on a column that is not returned; null values sort last in both directions. `limit` bounds rows; `budget` bounds *tokens* (Section 3.4); `after @t-N` is a read barrier against a write position (a no-op in the single-process engine, but parsed and honored so agent code is forward-compatible). Pseudo-columns available on any node binding: `.class`, `.id`, `.state`, `.health`, `.lineage`, `.query_traffic`.

3.4 Results, budgets, and continuation

No existing query language knows its consumer has a context window. GraphLang treats the serialized result as part of the language. Every result opens with a declared count the model can check itself against:

```
results: 20 of 47, budget hit
columns: sups.name, n_parts
VoltaChem, 12
...
truncated: 27 more. resume with: continue @c1a4
```

Overflow truncates at a row boundary and registers a **continuation handle**; `continue @c1a4 budget 1500 tokens` drains the remainder. Unstated budgets default to 2000 tokens. When a bare node binding is returned, the node prints in **incident encoding**, each node once with its edges gathered immediately around it, which is the format the evidence says models read most reliably:

```
part "lithium cell" {unit_cost: 4.2}
  supplied_by -> supplier "VoltaChem"
  uses <- product "PowerBank Pro"
```

3.5 Writes and receipts

Writes are the novel half of the surface. Every write verb states its provenance and returns a receipt.

`assert` creates a node or an edge:

```
assert part {name: "graphene sheet", unit_cost: 12.0}
  source doc:ingest-2026-08-24/p3 as gs
assert edge supplied_by(item: gs, source: #s-1)
  source doc:ingest-2026-08-24/p3
```

```
receipt: created part gs = #p-8 at @t-41
guards: schema ok, class invariants ok
dup candidates: 1
  #p-3 part "graphene sheets" score 0.95
    name 0.95; shared: unit_cost 12.0
```

```
resolve with: merge / distinct
```

The receipt carries the write's position (`@t-41`), the guards that ran, and any duplicate candidates the synchronous blocking stage found. Edge assertions name both roles explicitly and are type-checked against the schema, honoring subclass relationships.

merge resolves two nodes into one, with an explicit reconciliation policy:

```
merge gs, #p-3 prefer newest
```

The older node survives; the absorbed id becomes a permanent alias; the absorbed node's live edges are re-homed onto the survivor; and lineage records the complete pre-merge state of both nodes, so any merge can be unwound. **prefer newest** (the default) resolves property conflicts toward the newer node; **prefer source doc:...** prefers the node with the stated provenance and errors if neither has it, rather than silently picking one.

distinct is the durable opposite: a recorded assertion that two candidates are different, which suppresses the pair from every future candidate list.

```
distinct gs, #p-3 reason "sheet vs pre-cut sheets, different SKUs"
```

retire is temporal invalidation, not deletion: the node keeps its history and lineage but leaves current reads. **flag** is the explicit query-feedback verb; it increments the target's health record:

```
retire #s-3 reason "acquired by VoltaChem 2026-05, see #s-1"  
flag #p-8 reason "returned as Li-ion supplier, actually LiFeP04"
```

3.6 Granularity: refine and compact

An agent ingesting a report can store an entire table as one **blob** node (cheap, lossless), and only later, when queries start hitting it, split it:

```
assert table_blob {title: "EU supplier prices", payload: attach:csv-4471}  
  source doc:report-2026-08 as prices  
refine prices into part  
  with {name: col "component", unit_cost: col "eur_unit"}  
  as price_parts
```

```
receipt: refined #b-1 -> 142 part nodes at @t-88  
  lineage: each new node carries origin #b-1  
  dup candidates: 17 (queued per class)  
  blob state: composite, retained as lineage parent (cold)
```

The mapping is agent-supplied (column name to property, with values coerced to the class's declared types); every child records its origin; new children are dedup-checked against the existing graph but not against each other. **compact** is the inverse: it replaces a bound node set with a single summary node, retiring the members with lineage back to the summary.

3.7 Schema evolution: derive class

Agents extend the schema through the language, gated:

```
derive class broker from supplier with {takes_inventory: bool}
```

```
receipt: class broker provisional at @t-95
  quota: 500 instances; promotion needs stable use + dedup review
  note: similar existing class "distributor" (score 0.81), review advised
```

The new subclass inherits its base's properties, is immediately usable, and is quota-limited while provisional. Promotion gating is deferred beyond v0, but provisional status and quotas are enforced, and the receipt warns when a similar class already exists.

3.8 Deduplication pipeline

The system detects; the agent resolves. Detection is two stages. The **blocking stage** runs synchronously on every assert: candidates share a class and a blocking key (the first four characters of the casefolded, alphanumeric-only name), and are scored with a string-similarity ratio boosted to at least 0.9 when one normalized name is a strict prefix of the other ("Ionix Co" vs "Ionix"); pairs scoring at least 0.85 land in the receipt and the ledger. The **sweep stage** has the asynchronous shape of the planned embedding stage: it scans cross-block near-names per class and feeds the same ledger. The ledger is ordinary queryable data:

```
find dup_candidates where class = supplier order by score as dups
return dups.a, dups.b, dups.score limit 10
```

Merged and distinct-marked pairs are filtered out at read time, and **distinct** pairs are suppressed even on replay.

3.9 Health and the maintenance loop

Every node carries four health subscores in $[0,1]$, deliberately not combined into one magic number:

health.loss	construction instability: conflicting re-assertions of the same property ($\min(1, \text{conflicts}/5)$)
health.query	agent-reported failures via flag ($\min(1, \text{flags}/3)$)
health.structure	the max of supernode degree ($\text{deg}/100$), blob-payload traversal pressure ($\text{traversals}/50$), and an orphan floor (0.3 when degree is zero)
health.staleness	write positions since last confirming write, $/1000$

Health is queryable like everything else, so maintenance is not a mode; it is the same verbs pointed at a worklist:

```
find nodes where health.structure > 0.7 as worklist
return worklist.name, worklist.health limit 10
```

Read traffic feeds the structure signal automatically: each traversal durably increments the touched nodes' counters, with blob-payload traversals counted separately.

3.10 Verification and errors

Every program is verified whole before anything executes. The verifier walks the statements once, tracking a typing environment that maps each binding to its class (or to group, aggregate-value, or special-target types). It checks that classes, edge types, roles, and properties exist (including inherited properties); that **follow** sources are bound and arrival roles are type-compatible; that

aggregates consume groups or plain bindings but never other aggregates; that names are not rebound within a request; and that write references resolve. Errors are a language feature, engineered for self-repair:

```
error: unknown class "vendor" in line 1
  did you mean one of: part, product, supplier, table_blob?
nothing was executed.
```

Suggestions come from string proximity when a close name exists and from exhaustive listing when the option set is small. The closing sentence is load-bearing: the model knows no partial state exists.

3.11 Sessions and binding lifetime

Agents interact through a session that persists across requests. Within one request, bindings flow freely between reads and writes (a **find** can feed a **compact**; an **assert**'s name can appear in a later **merge** on the same request). Across requests, write arguments remain valid (the session keeps their node ids), but a read pipeline must re-establish its own bindings: the binding table is per-request, and the verifier rejects a stale read reference with a corrective error rather than executing against an empty table. Names can be re-bound in later requests. This rule exists because the alternative, discovered during adversarial review, was the worst kind of failure: a well-formed, silently empty result.

4 Implementation

4.1 Architecture

The core is 2,360 lines of dependency-free Python (standard library only); the test suite is 1,137 lines across 108 tests; the evaluation harness is a further 828 lines and is the only place with third-party dependencies (matplotlib for the figure).

Module	Responsibility	Lines
<code>parser.py</code>	tokenizer, logical lines, recursive descent	479
<code>ast_nodes.py</code>	one dataclass per statement	150
<code>verifier.py</code>	whole-program checking, typing env, suggestions	308
<code>schema.py</code>	classes, roles, inheritance, rendering	77
<code>engine/storage.py</code>	WAL, snapshots, replay, aliases, lineage	212
<code>engine/executor.py</code>	binding table, serialization, budgets	527
<code>engine/writes.py</code>	write verbs, receipts, reconciliation	343
<code>engine/dedup.py</code>	blocking keys, similarity, ledger	93
<code>engine/health.py</code>	the four subscore formulas	20
<code>session.py</code>	facade: parse, verify, route, binding lifetime	101
<code>cli.py</code>	file runner and REPL	45

4.2 Parser

A single regular expression tokenizes each logical line into strings, numbers, node ids, position and continuation tokens, provenance references, words, operators, and punctuation. Logical-line assembly joins indented physical lines and strips comments before tokenization. Parsing is recursive descent with one function per verb and no lookahead beyond a single token; unknown verbs, wrong keyword choices, and malformed clauses raise a **ParseError** carrying the logical line's starting line number. The parser enforces grammar strictness deliberately: bare words are

literals only in condition right-hand sides (where `class = supplier` needs them), `doc:` provenance is rejected as a property value, column paths are capped at three non-empty segments, and edge assertions take exactly two roles.

4.3 Verifier

The verifier is a single pass with a typing environment, as described in Section 3.10. Two details matter for agents. First, the environment distinguishes fresh bindings from `prior:-`marked bindings carried over from earlier session requests; reads reject the latter, writes accept them. Second, every verified statement is snapshotted with its environment into a `Plan`, which is the unit the executors consume, so execution never re-derives typing.

4.4 Storage engine

The store keeps the disaggregated architecture in one process. Durable truth is an append-only write-ahead log (`wal.jsonl`, one JSON record per write; the position is the record's index) plus immutable snapshot runs. Opening a store replays the newest run and then the WAL through the same record-application code path that live writes use, so replayed and live state cannot diverge. A snapshot serializes the full state as replayable records, writes `runs/run-<pos>.json`, and truncates the WAL. The record vocabulary is small and pure data: `put_node`, `patch_node`, `put_edge`, `retire`, `retire_edge`, `alias`, `lineage`, `distinct`, `dup`, `flag`, `traffic`. Nodes are stored with incident edge lists, so incident-format serialization is nearly a copy. Alias resolution chases chains with cycle protection; id counters are rebuilt from replayed ids so identifiers never collide across restarts. A bulk-append path loads benchmark-scale graphs (23,000 records) in about a hundred milliseconds.

4.5 Read executor

The executor materializes the binding table as a list of row dictionaries. `find` filters the node set (or the dedup ledger, or the schema) under the condition evaluator; `follow` expands rows via incident edge lists with alias resolution, retirement filtering, the trail-semantics edge-set check, and durable traffic recording; grouping is lazy (declared by `group by`, materialized by the first aggregate, which collapses rows into group rows carrying their members); `compute` evaluates per row; `return` sorts (nulls last in both directions), limits, serializes, and enforces the token budget, registering continuation payloads in the session's read context. Token accounting uses a documented heuristic of one token per four characters. The same pipeline backs a machine-readable entry point (`execute_rows`) that the evaluation harness uses for scoring, bypassing serialization.

4.6 Write executor, dedup, health

Each write verb is a function from a verified statement and a write context (store, schema, binding environment) to a `Receipt`. Receipts render in the exact format the language documentation shows, because agents pattern-match on receipts as much as on results. The merge implementation orders survivor and absorbed by creation position, records both pre-merge states in lineage before touching anything, re-homes edges through retire-and-recreate records, and only then installs the alias and reconciles properties. Refine parses CSV attachments at assert time into the blob's payload rows, coerces mapped columns to declared property types, and links every child to its origin. The dedup module and health formulas are exactly as specified in Sections 3.9 and 3.10; both are deliberately small (93 and 20 lines) because v0 stands in the string-similarity

and counter-based versions for the planned embedding and statistics pipelines while keeping the interfaces.

4.7 Session and CLI

The session facade parses, verifies with the persistent typing environment, routes statement runs to the read or write executor, threads read bindings into the write environment (so `compact` can consume a `find`), and converts every error class into agent-readable text ending in `nothing was executed.` where applicable. The CLI runs `.gl` files or an interactive REPL against a database directory.

5 Development methodology

Every component was built test-first: failing tests, minimal implementation, green suite, commit. On top of that, each stage passed through an **adversarial review loop**: a separate reviewer agent, briefed to hunt for silent wrong results and to verify every suspicion by executing repro snippets against the actual code, reviewed each stage; every confirmed finding was fixed with a regression test before work continued.

Three rounds produced thirteen confirmed findings: one P0, five P1, seven P2. A selection, with their classes:

- **Silent empty results (P0).** A read binding from a previous session request passed verification but executed against an empty table, returning a well-formed empty result. Fixed by the stale-binding rule of Section 3.12.
- **Grammar leaks (P1, P2).** The parser accepted bare words as property literals, `doc:` provenance as a value, unbounded column paths, and one- or three-role edge assertions. Each is now a parse error; each got a regression test.
- **Unfair benchmark accounting (P0, harness).** The token-economy metric compared GraphLang’s fully rendered output against a bare Python repr of Cypher rows, a threefold formatting penalty. Both conditions now count the complete textual result their agent would receive.
- **Null comparison hole (P1).** `compute same` on two unset properties returned true. It now raises a corrective error.
- **Merge policy fallback (P1).** `prefer source` silently preferred the second node when neither matched the stated source. It now errors.

The reviewer also independently re-verified each fix. The final suite is 108 tests, all green, on the merged mainline.

6 Evaluation

6.1 Setup

The harness draws on CypherBench [arXiv 2412.18702], whose NBA property graph (4,327 entities, 18,991 relations) ships with natural-language questions, gold Cypher, and gold answers. The graph is loaded identically into the GraphLang engine (via a deterministic schema derivation: entity labels become classes, relation labels become edge types, role names come from

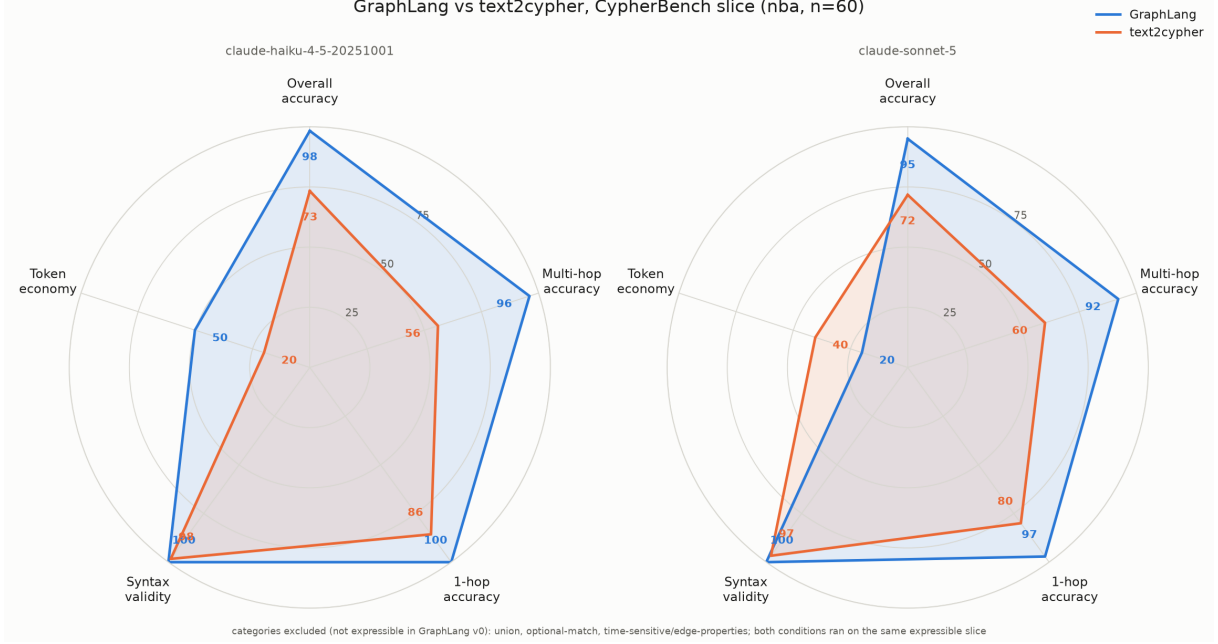


Figure 1: Both conditions, both models, five axes. Token economy is the normalized inverse of mean result tokens.

the endpoint class names) and into Neo4j 5 in Docker. Sixty questions are drawn with a fixed seed, stratified across the question categories v0 can express; union, optional-match, and edge-property categories are excluded, and *both conditions run on the identical slice*. Twenty-five of the sixty are multi-hop.

The two conditions are symmetric by construction. Same model per run (Claude Haiku 4.5, then Claude Sonnet 5). Same single repair retry on execution error, with the error text fed back. Same query extraction applied to model output. Same scoring: normalized, order-insensitive set comparison of executed rows against gold answers. GraphLang is prompted with a compact grammar, ten worked examples, and the live schema; the language has zero training corpus. Text-to-Cypher is prompted with the benchmark’s schema format and executes on Neo4j.

6.2 Results

Condition	Overall	Multi-hop	1-hop	Syntax valid	Result tokens
GraphLang + Haiku 4.5	98.3%	96.0%	100%	100%	245
text2cypher + Haiku 4.5	73.3%	56.0%	85.7%	98.3%	394
GraphLang + Sonnet 5	95.0%	92.0%	97.1%	100%	278
text2cypher + Sonnet 5	71.7%	60.0%	80.0%	96.7%	207

The published full-test-set frontier numbers (61.6% for Claude 3.5 Sonnet, 60.2% for GPT-4o on CypherBench; 51–58% for Claude Opus 4.8 and GPT-5.5 on the 2026 Text2GraphQuery-Bench) frame the scale but are on different benchmarks, not this slice. The clean comparison is within-slice and same-model, and its sharpest line is multi-hop with the small model: 96.0% against 56.0%. This is the Anka prediction realized on a graph workload: a designed language with zero corpus, taught in context, beating the most-documented graph language by forty points exactly where composition depth makes degrees of freedom expensive. Notably, Haiku with GraphLang also outperforms Sonnet with Cypher, which inverts the usual model-size economics.

6.3 The evaluation fed the language

Three spec-level changes came directly out of miss analysis, which is the loop working as intended:

1. **Global aggregates.** “How many unique venues are there?” forced models into a degenerate group-then-count that returned one row per venue. Aggregates over plain bindings now collapse the table.
2. **Trail semantics.** “Teams in the same division as X” included X under pure homomorphism; Cypher’s per-MATCH edge uniqueness excludes it. Per-row edge uniqueness aligned the semantics and the gold answers.
3. **The compute verb.** Difference and same-value questions were inexpressible. The word-operator compute line fits the one-line-one-name discipline and closed the category.

Alongside these, miss analysis produced engine fixes (nulls sorting first under descending order), harness robustness (prose-stripping query extraction applied to both conditions), and tutorial discipline rules (return exactly what is asked; use full entity names; dates compare as ISO strings).

6.4 Error analysis and threats to validity

The three residual GraphLang misses across both model runs are model discipline errors, not engine defects: an extra return column on a “who weighs more” question, a count of the wrong binding, one empty extraction. Threats to validity, all recorded in the results file: the slice covers only v0-expressible categories, so the headline number does not speak to union or edge-property workloads; it is one domain graph; set-based scoring collapses duplicate rows for both conditions equally; and token economy is not uniformly a GraphLang win (Sonnet’s Cypher wrote smaller raw results than GraphLang’s budgeted, header-carrying format on this slice). The forty-point multi-hop gap is far outside these error bars.

7 Will better models not simply fix Cypher?

The obvious objection to this whole project is that model progress will dissolve it: wait a generation or two, and frontier models will write Cypher well enough that a new language is pointless. The objection deserves a direct answer, because the evidence, ours and the field’s, runs the other way.

7.1 Scaling has had two years and has not delivered

Between late 2024 and mid 2026 the frontier advanced from Claude 3.5 Sonnet and GPT-4o to Claude Opus 4.8 and GPT-5.5, several major model generations. Cypher generation did not move with it. The February 2026 Text2GraphQuery-Bench evaluates exactly these newest models and reports Cypher execution accuracy of 51.2% for Claude Opus 4.8 and 53.3% for GPT-5.5 zero-shot, rising only to 57.6% and 55.3% with few-shot prompting [arXiv 2602.11745]. On its hardest query tier, GPT-5.5 falls to 19.1% zero-shot, and Claude Opus 4.8 reaches just 43.8% even with examples in context. Those are the same numbers, on the same task family, that motivated this project two model generations earlier. Our own benchmark is a second, live data point: on our slice, text-to-Cypher with Sonnet 5 (71.7%) is no better than with the far smaller Haiku 4.5 (73.3%), while switching the *language* moved both models by twenty-plus

points overall and thirty-plus on multi-hop. Model scaling is flat on this task; language design is not. And a working measure of the field’s own expectations: dedicated Cypher repair-and-triage systems were still being published in June 2026 [CYGNET, arXiv 2606.04645], which is not what an about-to-be-solved problem looks like.

7.2 The reason it stays flat is structural: corpus, not capacity

Text2GraphQuery-Bench’s sharpest finding is that a fine-tuned 8-billion-parameter model matches or exceeds the newest frontier models zero-shot on graph query generation, concluding that unfamiliarity with the language, not model capacity, is the barrier. Accuracy across query languages tracks training-corpus size, not language quality [arXiv 2411.05521], which is why GQL, the new ISO standard, benchmarks *worse* than Cypher despite being a cleaner language. This has a consequence the objection misses: even a model that mastered every Cypher idiom on GitHub has never seen *your* schema. Production schemas are out-of-corpus by definition, and schema hallucination is a per-deployment problem no amount of pretraining removes. A June 2026 line of work attacks this by generating synthetic training data per schema [arXiv 2606.14325], which concedes the point: each new graph needs its own supervision. GraphLang’s answer is structural rather than statistical: the vocabulary of legal classes, edges, roles, and properties is derived from the live schema and enforced at verification (and, in future, at decoding), so day-one correctness on an unseen schema is a property of the system, not of the corpus.

7.3 When syntax errors fall, the surviving errors are the dangerous ones

The same benchmark documents that as supervision increases, syntax errors drop sharply while “error mass shifts toward semantic and logical categories”: aggregation mistakes, nested-aggregation confusion, misuse of DISTINCT, schema-linking and filter errors. This is the worst possible failure profile for an autonomous agent, because a semantically wrong Cypher query is usually still *valid*: it executes, returns plausible rows or a confident empty set, and nothing downstream knows. Better models shrink the visible errors and leave the silent ones. GraphLang is aimed at precisely this residue: implicit grouping is inexpressible, identity-versus-value grouping is a spelling difference, direction does not exist, aggregation is staged and named, and the verifier converts an entire class of would-be silent failures into corrective errors that state **nothing was executed**. A model that never makes a syntax error still benefits from a language in which the remaining mistakes are loud.

7.4 A perfect Cypher generator still lacks the agent surface

Suppose the objection wins completely and some future model writes flawless Cypher. What it writes flawless Cypher *against* is a database that answers “1 row affected.” Nothing in Cypher, GQL, or their engines provides what agent construction workloads need: provenance-carrying writes, receipts that surface duplicate candidates at write time, an explicit merge/distinct resolution dialogue, lineage that makes merges and refinements unwindable, granularity verbs, temporal retirement, gated schema evolution, queryable health, or token-budgeted results with declared counts and continuations. The 2026 agent-memory landscape confirms nobody else is building this surface: production systems still ingest episodically through LLM extraction pipelines, and Mem0 *removed* its graph variant after finding it lost on recall while running three times slower at twice the token cost. Those failures are write-surface and economics failures, not generation failures, and model quality does not touch them. The same holds for the read side’s economics: an identical knowledge graph costs 2,645 tokens as an edge list and 13,503 as JSON-LD [arXiv 2504.07087], and accuracy collapses with textual distance between related facts [arXiv 2410.01985]; what the database *prints* is a property of the database.

7.5 Even for a perfect generator, the economics invert

Our results show a small, cheap model using GraphLang outperforming a frontier model writing Cypher (98.3% vs 71.7%). Agent fleets run on small models because they issue thousands of queries; a language that moves reliable graph access down-market is worth more than a frontier model that matches it at many times the price. Canonical forms add a further compounding benefit that improves as generation improves: when there is exactly one way to write each operation, correct agents produce byte-identical queries, which makes plan caching trivial and makes agent behavior auditable, the property the verification line of work needs. The better the models get, the more of their output can be cached, verified, and trusted, but only in a language canonical enough to make equality meaningful.

8 Limitations and future work

v0 defers, in language order: union and optional traversal; edge properties (the largest expressiveness gap against CypherBench, excluding the time-sensitive category); constrained decoding (the schema-closed grammar exists at verification time; wiring it into the decoder makes invalid labels unemittable rather than merely caught); class promotion gating; historical reads of retired nodes. In engine order: the object-storage substrate (SlateDB-based, per design decision 15), per-shard consistency with causal read tokens (16, the receipts already carry positions), partitioning (17), and embedding-based dedup with per-class vector indexes (18, the pipeline shape and ledger are already in place). The falsification bar the research phase set for the language (beat text-to-Cypher in-context on multi-hop slices before building infrastructure) is cleared, which is what those engine decisions were gated on.

A Complete grammar

```
program      := statement*
statement    := read_stmt | write_stmt | "schema"

read_stmt    := find | follow | group | aggregate | compute | return | continue
find         := "find" target ["where" cond] ["order" "by" col ["desc"]] "as" NAME
target       := CLASSNAME | "nodes" | "dup_candidates" | "class"
follow       := "follow" NAME EDGENAME ROLENAME ["where" cond] "as" NAME
group        := "group" "by" col "as" NAME
aggregate    := AGGVERB ["distinct"] col "as" NAME
              ; AGGVERB: count | sum | avg | min | max
compute      := "compute" col COMPOP col "as" NAME
              ; COMPOP: plus | minus | times | over | same
return       := "return" col ("," col)* ["order" "by" col ["desc"]]
              ["limit" INT] ["budget" INT "tokens"] ["after" POSITION]
continue     := "continue" HANDLE ["budget" INT "tokens"]

write_stmt   := assert_node | assert_edge | merge | distinct_s | refine
              | compact | retire | flag | derive
assert_node  := "assert" CLASSNAME props ["source" PROVENANCE] "as" NAME
assert_edge  := "assert" "edge" EDGENAME "(" ROLENAME ":" ref ","
              ROLENAME ":" ref ")" ["source" PROVENANCE]
merge        := "merge" ref "," ref ["prefer" ("newest" | "source" PROVENANCE)]
distinct_s   := "distinct" ref "," ref "reason" STRING
refine       := "refine" ref "into" CLASSNAME "with" mapping "as" NAME
compact      := "compact" NAME "as" NAME props
retire       := "retire" ref "reason" STRING
flag         := "flag" ref "reason" STRING
derive       := "derive" "class" NAME "from" CLASSNAME "with" propdecls
```

```

cond      := clause (("and" | "or") clause)*      ; "and" binds tighter
clause    := col OP literal                        ; OP: = != > >= < <= contains
col       := NAME ["." NAME ["." NAME]]           ; 1..3 non-empty segments
props     := "{" NAME ":" literal ("," NAME ":" literal)* "}"
mapping    := "{" NAME ":" "col" STRING ("," ...)* "}"
propdecls := "{" NAME ":" TYPENAME ("," ...)* "}" ; TYPENAME: str|int|float|bool
literal   := STRING | NUMBER | "true" | "false" | ATTACH
           ; bare words allowed only as condition right-hand sides
ref       := NAME | NODEID
NODEID    := "#" [a-z]+ "-" [a-z0-9]+
POSITION  := "@t-" [0-9]+
HANDLE    := "@c" [a-z0-9]+
PROVENANCE := ("doc:" | "attach:") [^ ]+

; One statement per logical line. Indented physical lines continue the
; previous logical line. Comments are "# " (hash space) or an unindented
; "#" line; node ids on continuation lines are never comments.

```

B A construction session, end to end

> lines are the agent; green boxes are the system. This transcript is the shape the integration tests replay.

```
> schema
```

```

classes:
  product{name, launch_year}
  part{name, unit_cost}
  supplier{name, country}
  table_blob{title, payload}
edges:
  uses(whole: product, component: part)
  supplied_by(item: part, source: supplier)

```

```
> assert supplier {name: "Ionix Co", country: "KR"}
    source doc:report-08/p1 as ionix
```

```

receipt: created supplier ionix = #s-1 at @t-1
guards: schema ok, class invariants ok

```

```
> assert supplier {name: "Ionix"} source doc:report-08/p2 as ionix2
```

```

receipt: created supplier ionix2 = #s-2 at @t-2
dup candidates: 1
  #s-1 supplier "Ionix Co" score 0.9
resolve with: merge / distinct

```

```
> merge ionix, ionix2 prefer newest
```

```

receipt: merged -> #s-1 (lineage keeps both) at @t-5
absorbed #s-2 is now an alias of #s-1
policy: prefer newest

```



```
> assert table_blob {title: "Ionix price list Q3", payload: attach:q3}
    source doc:report-08/p4 as pricelist
> refine pricelist into part
    with {name: col "item", unit_cost: col "unit_eur"} as rows
```

```
receipt: created table_blob pricelist = #t-1 at @t-7
receipt: refined #t-1 -> 2 part nodes at @t-10
    lineage: each new node carries origin #t-1
    blob state: composite, retained as lineage parent (cold)
```

```
> find part as p
> return p.name, p.unit_cost order by p.unit_cost
```

```
results: 2 of 2, complete
columns: p.name, p.unit_cost
anode, 2.5
cathode, 3.5
```

The loop closes: writes surface duplicates, health surfaces grain problems, the same small verb set fixes both, and every step traces through lineage to the document it came from.